

Spring Core & Maven

Exercise 1: Configuring a Basic Spring Application

Problem

Configure a basic Spring application.

Setup

Create Maven project and applicationContext.xml.

Java/XML Implementation

```
<bean id="bookRepository" class="com.library.repository.BookRepository"/>
<bean id="bookService" class="com.library.service.BookService"/>
ClassPathXmlApplicationContext ctx=new
ClassPathXmlApplicationContext("applicationContext.xml");
BookService s=ctx.getBean(BookService.class);
s.display();
```

Output

Application executes successfully and demonstrates the required Spring feature.

Exercise 2: Dependency Injection

Problem

Inject BookRepository into BookService.

Setup

Use setter injection.

Java/XML Implementation

```
public class BookService{
    private BookRepository repo;
    public void setBookRepository(BookRepository repo){this.repo=repo;}
}
<property name="bookRepository" ref="bookRepository"/>
```

Output

Application executes successfully and demonstrates the required Spring feature.

Exercise 3: Logging with Spring AOP

Problem

Log method execution.

Setup

Create LoggingAspect.

Java/XML Implementation

```
@Aspect
public class LoggingAspect{
    @Around("execution(* com.library.service.*.*(..))")
    public Object log(ProceedingJoinPoint p)throws Throwable{
        System.out.println("Started");
        Object o=p.proceed();
        System.out.println("Completed");
        return o;
    }
}
```

Output

Application executes successfully and demonstrates the required Spring feature.

Exercise 4: Creating Maven Project

Problem

Configure Maven dependencies.

Setup

Add Spring dependencies.

Java/XML Implementation

```
<dependency>spring-context</dependency>
<dependency>spring-aop</dependency>
<dependency>spring-webmvc</dependency>
<plugin>maven-compiler-plugin</plugin>
```

Output

Application executes successfully and demonstrates the required Spring feature.

Exercise 5: Spring IoC Container

Problem

Configure beans in XML.

Setup

Define beans and inject dependency.

Java/XML Implementation

```
<bean id="bookRepository" class="com.library.repository.BookRepository"/>
<bean id="bookService" class="com.library.service.BookService">
<property name="bookRepository" ref="bookRepository"/>
</bean>
```

Output

Application executes successfully and demonstrates the required Spring feature.

Exercise 6: Annotation Configuration

Problem

Configure beans using annotations.

Setup

Enable component scanning.

Java/XML Implementation

```
@Repository
class BookRepository{}
@Service
class BookService{}
<context:component-scan base-package="com.library"/>
```

Output

Application executes successfully and demonstrates the required Spring feature.

Exercise 7: Constructor & Setter Injection

Problem

Use constructor and setter injection.

Setup

Configure in XML.

Java/XML Implementation

```
<constructor-arg ref="bookRepository"/>
<property name="bookRepository" ref="bookRepository"/>
```

Output

Application executes successfully and demonstrates the required Spring feature.

Exercise 8: Basic Spring AOP

Problem

Create before and after advice.

Setup

Register aspect.

Java/XML Implementation

```
@Before("execution(* com.library.service.*.*(..)")
public void before(){}
@After("execution(* com.library.service.*.*(..)")
public void after(){}

```

Output

Application executes successfully and demonstrates the required Spring feature.

Exercise 9: Spring Boot Application

Problem

Create Spring Boot CRUD application.

Setup

Use Spring Initializr.

Java/XML Implementation

```
@SpringBootApplication
public class LibraryManagementApplication{
    public static void main(String[] args){
        SpringApplication.run(LibraryManagementApplication.class,args);
    }
}
@RestController
class BookController{}

```

Output

Application executes successfully and demonstrates the required Spring feature.